錯誤控制編碼期中 Project

《Decoding Process of Convolutional Codes》

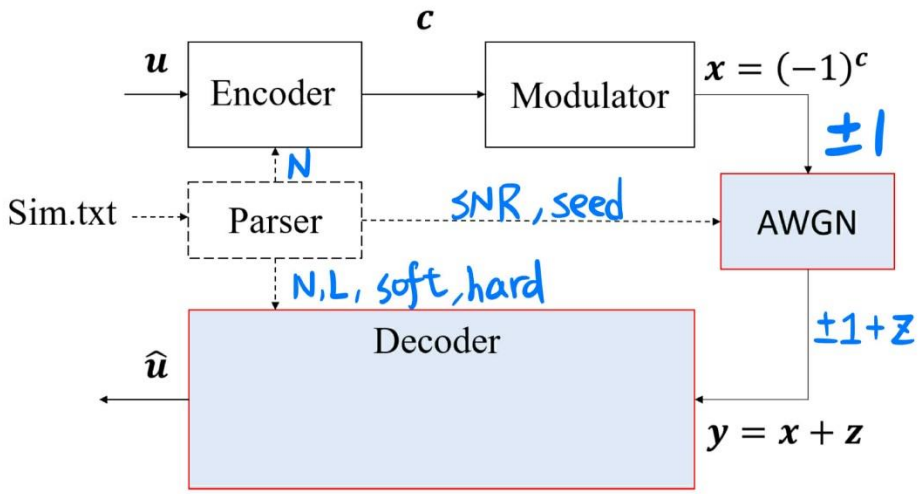
學生：陳品維

學號：E2410286

Project Outline

1. 系統架構圖
2. 程式流程解釋
3. 參數設定
4. 模擬數據及表格
5. 模擬圖及 coding gain at BER = 10^{-4}
6. 實作心得與問題討論

(1) 系統架構圖



$$\boldsymbol{G}(D) = \begin{pmatrix} 1 + D^2 + D^3 + D^5 + D^6 & 1 + D + D^2 + D^3 + D^6 \end{pmatrix}.$$

以上為本次專題主要的系統方塊圖。首先 u 為 information bits。u 會先送進去 encoder 進行編碼的動作。編碼的方式主要是利用輸入訊號跟前幾筆訊號的關聯性(time delay 的特性)來進行，透過 G(D)的 Generator Matrix 去進行實現。從 Generator Matrix 中可以發現 code rate = 1/2。接著將編碼好的訊號送進去 BPSK modulator 去進行 BPSK mapping，把 0/1 轉成 1/-1 的形式。接著會利用數學模型生成出類似 AWGN Channel，並且將 mapping 完的訊號加上雜訊，最後進行 decode 的動作。

(2) 程式流程解釋

AWGN Channel 相關參數設定，會用於 ran1()及 normal()兩個函式

```
#define IA 16807
#define IM 2147483647
#define AM (1.0 / IM)
#define IQ 127773
#define IR 2836
#define NTAB 32
#define NDIV (1 + (IM - 1) / NTAB)
#define EPS 1.2e-7
#define RNMX (1.0 - EPS)
```

用於計算程式執行全程所需時間，建立於程式開頭及程式結尾

```
// Start the timer to measure the execution time of the simulation
clock_t start, end;
double cpu_time_used;
start = clock();
```

讀取 Sim.txt 內部輸入參數值(序列長度、窗口、訊雜比、亂數種子、決策)

(設定為 N, win_length, SNR, SEED, decision)

```
ifstream file("Sim.txt");
if (!file.is_open()) { ... }

// 讀取參數
int N, win_length, decision;
double snr, sigma;
long seed;
```

根據 decision 的種類決定所要輸出的檔案

```
string file_name = (decision == 0) ? "Output_Hard.txt" : "Output_Soft.txt";
ofstream output_file(file_name, ios::out);
```

依照特定的遞迴規則產生 u 序列生成 information bits

```
// initialize the sequence
vector<int> u(N + 6, 0);
u[0] = 1;
for (int i = 1; i < 6 && i < N; ++i) {
    u[i] = 0;
}
```

```
// the recursive sequence
for (int L = 0; L + 6 < N; ++L) {
    u[L + 6] = binary_add(u[L], u[L + 1]);
}
```

利用設定好的 generator matrix 生成 codewords 並進行 BPSK mapping

```
// 計算 c1 和 c2
int c1 = binary_add(d0, binary_add(d2, binary_add(d3, binary_add(d5, d6))));
int c2 = binary_add(d0, binary_add(d1, binary_add(d2, binary_add(d3, d6))));
// Antipodal mapping
int x1 = (c1 == 0) ? 1 : -1;
int x2 = (c2 == 0) ? 1 : -1;
```

用於生成 AWGN noise 的函式

```
// random number generator
double ran1(long *idum) { ... }
// normal distribution generator
void normal(double &n1, double &n2, double sigma, long *idum) { ... }
```

把 codewords 加上雜訊

```
// normal distribution noise
double n1, n2;
normal(n1, n2, sigma, idum);

// add the noise
double x1_with_noise = x1 + n1;
double x2_with_noise = x2 + n2;
```

初始化狀態的個數為 64

```
// define constant
const int NUM_STATES = 64; // (2^6) state quantity
const int MAX_BITS = 6;
```

建立狀態訊息結構(包含當下的輸入及輸出、先前的狀態以及累積路徑距離)

```
// define state information structure
struct StateInfo {
    int input;                  // The input bit leading to this state
    int prev_state;             // The previous state
    int output1, output2;       // Outputs associated with this state
    double path_metric;         // Accumulated path metric
};
```

建立 state diagram 並初始化 state diagram 的相關函式

```
// calculate output 和 next state
void calculate_next_state(bitset<MAX_BITS> current_state, int input, int &output1, int &output2,
int next_state_table[NUM_STATES][2];
int output_table[NUM_STATES][2][2];

// initialize state diagram
void initialize_state_tables() { ... }
```

接著是最核心的 viterbi decoding 函式。首先先從 state 0 出發，計算在每個時刻當下的 output1 及 output2 跟 receive signals 的累積歐基里德距離，累積到所設定到的窗口時(設定為 32)，開始進行回溯。回溯的方法選擇 previous state 中 metric 增加量最少者，回推到最前面的 bits 進行輸出。接著進行窗口滑移，滑移至下一個位置再次計算累積路徑距離，選擇路徑最小者開始回溯，若增加路徑相同，總選擇 previous state 編號較小者，依此類推重複以上步驟。

```
// Viterbi decoding
void viterbi_decode(const vector<pair<double, double>> &noisy_outputs, int win_length, vector<int> &decoded_inputs,
    size_t total_length = noisy_outputs.size();
    decoded_inputs.clear();
    decoded_outputs.clear();

    vector<vector<double>> metric(NUM_STATES, vector<double>(win_length, numeric_limits<double>::max()));
    metric[0][0] = 0.0; // Initial state "000000"

    vector<vector<int>> state_outputs;
    compute_state_diagram(decision, state_outputs);
```

使用 vector<int> decoded_inputs 及 vector<pair<int, int>> decoded_outputs 來記錄 decode 完的結果

```
// 解碼
initialize_state_tables();
vector<int> decoded_inputs;
vector<pair<int, int>> decoded_outputs;
viterbi_decode(noisy_outputs, win_length, decoded_inputs, decoded_outputs, decision);
```

計算 error 個數並計算 bit error rate

```
int error_count = 0;
for (size_t i = 0; i < u.size(); ++i) {
    if (u[i] != decoded_inputs[i]) {
        error_count++;
    }
}
double error_rate = static_cast<double>(error_count) / u.size();
```

紀錄為程式中止並結算程式執行時間

```
end = clock();
cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("running time: %f sec\n", cpu_time_used);
system("pause");
```

(3) 參數設定

所要讀取的檔案 Sim.txt 的內容如下所示第一行為 number of decode bits，第二行為 truncation window length，第三行為 SNR 數值，第四行為一個負數做為生成隨機亂數的種子，第五行為 decision 的種類。

```
1000000      %N: number of decoded bits
32     %L: truncation window length
5.0     %SNR: bit signal-to-noise ratio (dB)
-2023 %:Seed: a negative integer
1      %0=>Hard-decision; 1=>Soft-decision
```

(4) 模擬數據及表格

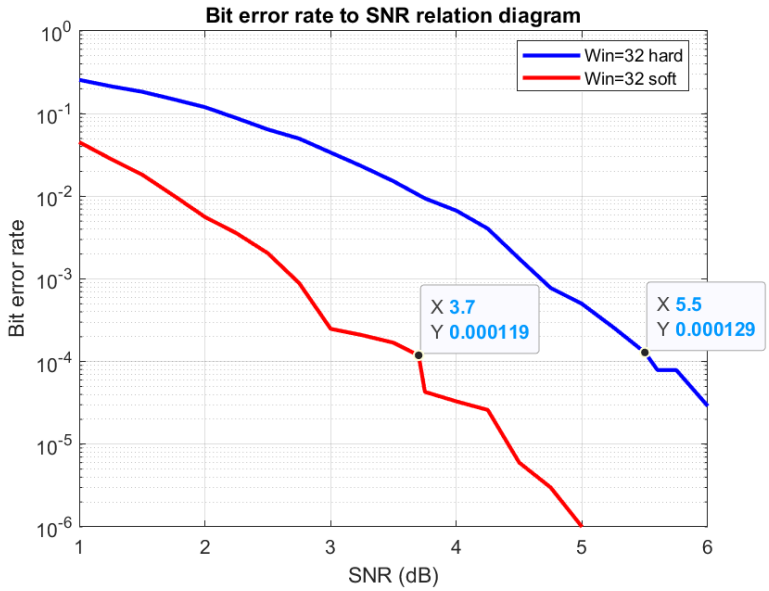
Truncation window = 32, hard decision

SNR(dB)	1.00	1.25	1.50	1.75	2.00	2.25	2.50
BER	0.25591	0.21432	0.18385	0.14912	0.11963	0.08832	0.06434
SNR(dB)	2.75	3.00	3.25	3.50	3.75	4.00	4.25
BER	0.04980	0.03382	0.02298	0.01519	0.00941	0.00669	0.004059
SNR(dB)	4.50	4.75	5.00	5.25	5.50	5.75	6.00
BER	0.00175	0.00077	0.00049	0.00025	0.00012	0.00007	0.00002

Truncation window = 32, soft decision

SNR(dB)	1.00	1.25	1.50	1.75	2.00	2.25	2.50
BER	0.04518	0.02836	0.01825	0.01022	0.00561	0.00357	0.00204
SNR(dB)	2.75	3.00	3.25	3.50	3.75	4.00	4.25
BER	0.00088	0.00024	0.00020	0.00016	0.00004	0.00003	0.00002
SNR(dB)	4.50	4.75	5.00	5.25	5.50	5.75	6.00
BER	0	0	0	0	0	0	0

(5) 模擬圖



根據模擬圖計算 coding gain at BER = 10^{-4} 大約為 1.8(dB)

(6) 實作心得與問題討論

(一) 在設計 viterbi decoding 演算法的過程中，記憶體的配置相當重要，記憶體如果配置不當，程式很容易跑不動或是直接當機。

(二) 在設計解碼器的過程中，有些程式內部邏輯可能還沒有到非常完善，在某些測資的錯誤率仍尚未助教設定的標準。

(三) 在計算邊界條件以及某些特殊情況時利用向量進行計算還不夠嚴謹